

Documentation — Phase 1: WHAT

Purpose. Define the solution architecture for Smart City Air Quality & Traffic. No code yet. The mentor reviews and signs off this document before implementation begins.

Team: 67

Role Name: Smart city air quality & traffic

Team lead: Khamidullokhon Abduvokhidov

Member 2: Madina Abdusattorova

Member 3: Osiyo Rashidova

Member 4: Abduloxunov Davlatyor

Date: Chosen scenario: Smart City Air Quality & Traffic

1. Problem Restatement

In your own words, restate the business problem in 3–5 sentences.

(write here)

A city has deployed a network of 50 air-quality sensors and 30 traffic counters across its road system, generating hundreds of thousands of readings over time. The core challenge is not simply measuring pollution, but understanding *causality with a time delay*: vehicle traffic generates emissions that take minutes to hours to disperse and register at nearby sensors, so a naive same-timestamp correlation will miss the relationship entirely. The goal is to identify which specific road's traffic count is the strongest leading indicator of which specific sensor's PM2.5 readings — and crucially, by how many minutes (the lag) that relationship holds. This insight would allow the city's traffic department to use live traffic data as an early-warning system for air quality events, enabling proactive interventions like rerouting heavy vehicles or triggering public health alerts before pollution peaks actually arrive at sensors.

2. Proposed Solution Stack

List the specific tools, libraries, and methods you plan to use at each stage.

2.1 Data download & ingestion

1. Download CSV file directly from Kaggle

2. Use `pandas.read_csv()` to read and edit data. Use `pd.concat()` to integrate all the 12 DataSets into one big DataFrame (DF) to make work easier
3. Use `.shape`, `.info`, `.dtypes`, `isnull().sum()`, and `.describe()` to see what is in the DF, and we will be using them throughout the cleaning process too to see the changes.

2.2 Data cleaning / hygiene

We will be using pandas for every data manipulation

1. Use `DF.dropna(axis=1, thresh=0.5*len(DF))` to drop all the columns with more than 50% the data Nan for accuracy purposes.
2. Identify outliers and long Nan values (namely 48-hour gaps). Use `.cumsum()` to calculate long Nan values
3. Fill acceptable missing values with `.mean()`, `.mode()` within the rows
4. Use `pd.Timedelta(hours=)` to bring all the time zones into one system across 12 datasets before merging them
5. Use `np.sin(np.radians(df.wind_deg))` and `np.cos(np.radians(df.wind_deg))` functions and replace `wind_deg` columns with `sin` and `cos` to make it possible to calculate comparatively to eastward and northward components.
6. Drop unused columns which are: `datetime`, `station`, `PM2.5`, `PM10`, `NO2`, `CO`, `temp`, `wind_speed`, `wind_deg`, `traffic_count`
7. Check and drop duplicated rows with `DF.duplicated(['datetime', 'station']).sum()`

2.3 Analysis / modelling

1. Reshape from long format (one row per station per hour) to wide format: one row per timestamp, one column per station's PM2.5, plus traffic columns. This enables vectorised lag operations.
2. Place the data in ranking order - sort all (road, sensor, lag)
3. Cross-correlation sweep - For every (road $\times$ sensor) pair, compute Pearson r at lags 0, 1, 2, ... 12 hours (traffic leading PM2.5). Record the best r and its lag. This is the core analytical method — `scipy.stats.pearsonr` inside a nested loop over roads, stations, and lags.
4. Validation - Confirm the winner is not a spurious diurnal artifact by partialling out hour-of-day using `scipy.stats.pearsonr`: for each lag, `pearsonr()` returns both the correlation coefficient r and a two-tailed p-value directly, letting us confirm that the traffic $\rightarrow$ PM2.5 relationship at the optimal lag is statistically significant ($p < 0.05$) and not a random coincidence. If the p-value remains below the threshold even after we restrict the data to a single wind-direction quadrant or a single season, the result is robust.

2.4 Visualization

1. For visualisation we will be using matplotlib
2. Matplotlib Heatmap (roads x sensors)
3. Scatterplot with trend line to answer - “does the relationship hold visually when lag is applied”
4. Np.polyfit dual-axis time series (72 h window)

2.5 Reporting

1. Analysis with `.ipynb` file
2. 1-page executive report with pdf file
3. Final dataframe with csv file

3. Logic Map

Stage 1 — Ingestion

- **Input:** 12 raw CSV files (e.g., `PRSA_Data_Aotizhongxin_20130301-20170228.csv`) covering the period from March 2013 to February 2017.
- **Action:** Use Python (Pandas) to load and concatenate all 12 site-specific files into a single master DataFrame.
- **Output:** A unified dataset containing hourly air pollutant and meteorological data for all monitoring sites.

Stage 2 — Cleaning

- **Input:** The unified raw DataFrame.
- **Action:** Handle missing values (NaN) in pollutant columns (PM2.5, PM10, etc.) using linear interpolation; combine `year`, `month`, `day`, and `hour` columns into a single `datetime` object.
- **Output:** A cleaned, time-series indexed dataset ready for statistical processing.

Stage 3 — Analysis

- **Input:** Clean dataset.
- **Action:** Compare air pollution levels between different stations (like Aotizhongxin, Changping, etc.). Also calculate Pearson correlation between weather variables (TEMP, DEWP, WSPM) and PM2.5.
- **Output:** Ranking of stations and statistical analysis of weather impact on pollution.

Stage 4 — Visualization

- **Input:** Analysis results.

- **Action:** Create faceted plots to show PM2.5 changes for all 12 stations. Create a scatter plot to show the relationship between temperature and pollution.
- **Output:** Visual charts that show air quality trends.

Stage 5 — Insight & Recommendation

- **Input:** Charts and analysis.
- **Action:** Identify which areas are most polluted and explain how wind speed (WSPM) affects pollution levels.
- **Output:** Environmental recommendations for Beijing city.

4. Workload Split (3 Developers)

Developer Responsibility Stages owned

Developer	Responsibility	Stages Owned
Developer1	Data Architecture: Combine 12 files, fix time format, and handle missing values (NaN).	Stage 1, Stage 2
Developer2	Statistical Analysis: Calculate the relationship between weather (Temperature, Wind) and pollution (PM2.5).	Stage 3
Developer3	Data Visualization: Use Seaborn/Matplotlib to create comparison charts between stations and prepare final conclusions.	Stage 4, Stage 5

5. Risks & Mitigation

List 3 things that could go wrong and your mitigation plan.

1. (Missing data) — (Fill gaps with nearby stations, don't delete rows)
2. (Sensors can't detect low pollution levels) — (Note there are low pollution levels, don't take them as a zero pollution)
3. (Air pollution patterns change) — (Analyze by season and year, don't mix them together)